

RAG Stage-1 Complete Development Roadmap

AI Grand Challenge PS-4 - 90 Day Sprint Plan

STAGE 1 OBJECTIVE

Build a **world-class document retrieval system** that maximizes:

- **Recall@k (50% weight)** - Find ALL relevant documents
- **Precision@k (20% weight)** - Minimize false positives
- **NDCG (30% weight)** - Perfect ranking of relevance

Success Target: Top 3 position in shortlisting to guarantee Stage-2 qualification

90-DAY DEVELOPMENT TIMELINE

WEEK 1-2: Foundation & Data Engineering (Days 1-14)

Day 1-3: Environment Setup

```
bash

# Core Development Stack
- Python 3.9+ with CUDA support
- Docker containers for reproducibility
- Git LFS for large dataset management
- Weights & Biases for experiment tracking
```

Day 4-7: Dataset Analysis & Preprocessing

Critical Tasks:

1. Download & Catalog Training Data (10GB)

- Cord-19: COVID research papers
- WikiHop: Multi-hop reasoning dataset
- HotpotQA: Complex question answering
- EU-Legislation: Legal documents
- APT NOTES: Security research
- RobustQA & MultiHop-QA: QA datasets

2. Data Quality Analysis

python

Key metrics to analyze

- Document length distribution
- Query complexity levels
- Answer span locations
- Multi-document reasoning requirements
- Language patterns and vocabulary

3. Preprocessing Pipeline

python

Document preprocessing

- Text cleaning & normalization
- Sentence segmentation
- Chunk size optimization (128-512 tokens)
- Metadata extraction
- Duplicate detection & removal

Day 8-14: Baseline System Development

Deliverables:

- Basic BM25 retrieval baseline
- Simple embedding-based retrieval
- Evaluation framework implementation
- Initial performance benchmarks

WEEK 3-6: Advanced Retrieval Architecture (Days 15-42)

Day 15-21: Multi-Stage Retrieval Pipeline

Architecture: Hybrid Retrieval System

python

Stage 1: Sparse Retrieval (BM25++)

- Enhanced BM25 with custom term weighting
- Domain-specific stopwords optimization
- Query expansion using WordNet/ConceptNet

Stage 2: Dense Retrieval

- Multiple embedding models ensemble
- Cross-encoder reranking
- Domain adaptation fine-tuning

Key Implementation:

1. Enhanced BM25 Implementation

python

```
from rank_bm25 import BM25Plus
# Custom term frequency weighting
# Document length normalization
# Query-specific parameter tuning
```

2. Dense Retrieval Models

python

```
# Primary: sentence-transformers/all-MiniLM-L12-v2
# Secondary: sentence-transformers/all-mpnet-base-v2
# Domain-specific: allenai/specter (for scientific docs)
# Legal-specific: nlpauieb/legal-bert-base-uncased
```

Day 22-28: Embedding Optimization

Multi-Embedding Strategy:

1. Domain-Specific Embeddings

- Scientific papers: SciBERT, BioBERT
- Legal documents: Legal-BERT
- General knowledge: MPNet, MiniLM

2. Embedding Fine-tuning

python

```
# Contrastive learning on domain data
# Hard negative mining
# In-batch negative sampling
# Cross-domain adaptation
```

3. Vector Database Optimization

```
python

# FAISS with HNSW indexing
# Approximate nearest neighbor search
# Index compression techniques
# Multi-index strategy for different domains
```

Day 29-35: Query Processing & Expansion

Advanced Query Understanding:

1. Query Analysis Pipeline

```
python

# Named entity recognition
# Intent classification
# Complexity scoring
# Multi-hop detection
```

2. Query Expansion Techniques

```
python

# Pseudo-relevance feedback
# Word embedding similarity
# Knowledge graph expansion
# Acronym/abbreviation handling
```

Day 36-42: Reranking & Fusion

Multi-Stage Reranking:

1. Cross-Encoder Reranking

```
python
```

```
# microsoft/DialoGPT-medium for reranking
# Custom cross-encoder training
# Query-document pair scoring
```

2. Score Fusion Strategy

```
python

# Weighted combination of:
# - BM25 scores (normalized)
# - Dense retrieval scores
# - Cross-encoder scores
# - Domain-specific boosting
```

WEEK 7-10: Performance Optimization (Days 43-70)

Day 43-49: Hyperparameter Optimization

Systematic Parameter Tuning:

1. Bayesian Optimization Framework

```
python

import optuna
# Optimize:
# - BM25 k1, b parameters
# - Embedding model combinations
# - Fusion weights
# - Chunk size and overlap
# - Top-k values for each stage
```

2. A/B Testing Framework

```
python

# Split validation data
# Statistical significance testing
# Performance regression detection
```

Day 50-56: Advanced Retrieval Techniques

Cutting-Edge Methods:

1. ColBERT-style Late Interaction

python

- # *Token-level similarity computation*
- # *Efficient approximate search*
- # *Better semantic matching*

2. Learned Sparse Retrieval

python

- # *SPLADE implementation*
- # *Sparse attention mechanisms*
- # *Vocabulary expansion learning*

3. Multi-Vector Dense Retrieval

python

- # *Multiple representations per document*
- # *Aspect-based retrieval*
- # *Fine-grained matching*

Day 57-63: Ensemble Methods

Retrieval Ensemble Strategy:

1. Model Ensemble

python

- # *5+ different retrieval models*
- # *Weighted voting mechanism*
- # *Dynamic weight adjustment*
- # *Confidence-based selection*

2. Feature Engineering

python

- # *Document metadata features*
- # *Query-document interaction features*
- # *Temporal features*
- # *Source reliability scores*

Day 64-70: Evaluation & Validation

Comprehensive Testing:

1. Cross-Validation Strategy

```
python

# 5-fold cross-validation
# Domain-stratified splits
# Temporal validation splits
# Query complexity stratification
```

2. Error Analysis Framework

```
python

# Failed case analysis
# Precision/Recall breakdown by query type
# Domain-specific performance analysis
# Computational efficiency profiling
```

WEEK 11-12: Final Optimization & Submission Prep (Days 71-84)

Day 71-77: System Integration

1. End-to-End Pipeline

```
python

# Dockerized deployment
# API endpoint creation
# Batch processing optimization
# Memory management
```

2. Performance Monitoring

```
python

# Real-time metrics tracking
# Latency optimization
# Resource usage monitoring
```

Day 78-84: Mock Dataset Testing

1. Mock Dataset Analysis (Released Day 45)

```
python
```

```
# Immediate performance evaluation
# Quick iteration on weak points
# Leaderboard position tracking
```

2. Final Model Selection

```
python

# Best performing ensemble
# Computational efficiency balance
# Robustness testing
```

WEEK 13: Shortlisting Preparation (Days 85-90)

Day 85-87: Final Testing

- Stress testing with large document sets
- Edge case handling verification
- Performance consistency validation

Day 88-90: Submission Ready

- Code cleanup and documentation
- Docker container optimization
- Submission format compliance
- Emergency fallback models ready

⚡ TECHNICAL ARCHITECTURE

Core System Components

```
python
```



```

class AdvancedRAGRetriever:
    def __init__(self):
        # Multi-stage retrieval pipeline
        self.sparse_retriever = EnhancedBM25()
        self.dense_retrievers = [
            SentenceTransformerRetriever("all-mpnet-base-v2"),
            SentenceTransformerRetriever("all-MiniLM-L12-v2"),
            DomainSpecificRetriever("scibert"),
            DomainSpecificRetriever("legal-bert")
        ]
        self.cross_encoder = CrossEncoderReranker()
        self.fusion_model = ScoreFusionModel()

    def retrieve(self, query, top_k=100):
        # Stage 1: Sparse retrieval
        sparse_results = self.sparse_retriever.search(query, k=1000)

        # Stage 2: Dense retrieval ensemble
        dense_results = []
        for retriever in self.dense_retrievers:
            results = retriever.search(query, k=500)
            dense_results.append(results)

        # Stage 3: Result fusion
        fused_results = self.fusion_model.combine(
            sparse_results, dense_results
        )

        # Stage 4: Cross-encoder reranking
        final_results = self.cross_encoder.rerank(
            query, fused_results[:200], top_k=top_k
        )

        return final_results

```

Performance Optimization Strategies

1. Computational Efficiency

```

python

# GPU acceleration for embedding computation
# Batch processing optimization
# Caching frequently accessed embeddings
# Quantization for reduced memory usage

```

2. Indexing Optimization

```
python

# FAISS with IVF indexing
# Product quantization
# Multi-GPU parallelization
# Incremental index updates
```

3. Memory Management

```
python

# Streaming document processing
# Lazy loading of embeddings
# Garbage collection optimization
# Memory-mapped file access
```

 EVALUATION STRATEGY

Target Metrics (Stage 1 Focus)

Recall@k (50% weight) - PRIMARY FOCUS

```
python

# Target: >0.95 Recall@100
# Strategy:
# - Cast wide net in initial retrieval (k=1000)
# - Multiple retrieval strategies
# - Query expansion techniques
# - Comprehensive document coverage
```

Precision@k (20% weight)

```
python

# Target: >0.85 Precision@10
# Strategy:
# - Strong reranking models
# - Relevance threshold tuning
# - False positive detection
```

NDCG (30% weight)

```
python
```

```
# Target: >0.90 NDCG@100
```

```
# Strategy:
```

```
# - Perfect ranking optimization
```

```
# - Graded relevance handling
```

```
# - Cross-encoder fine-tuning
```

Continuous Evaluation Pipeline

```
python
```

```
# Daily evaluation runs
```

```
# Performance regression detection
```

```
# Metric decomposition analysis
```

```
# A/B testing framework
```

SUCCESS METRICS & MILESTONES

Week 2 Milestone

- ☐ All datasets downloaded and preprocessed
- ☐ Basic BM25 baseline achieving >0.70 Recall@100
- ☐ Evaluation framework operational

Week 6 Milestone

- ☐ Multi-stage retrieval pipeline complete
- ☐ >0.85 Recall@100 achieved
- ☐ Domain-specific optimization implemented

Week 10 Milestone

- ☐ Advanced ensemble methods deployed
- ☐ >0.92 Recall@100, >0.80 Precision@10
- ☐ Mock dataset results in top 5

Week 13 Target

- ☐ **>0.95 Recall@100, >0.85 Precision@10, >0.90 NDCG@100**
- ☐ **Top 3 position guaranteed for Stage-2**
- ☐ Submission ready with fallback models

COMPETITIVE ADVANTAGES

1. Multi-Domain Expertise

- Specialized models for each dataset type
- Domain adaptation techniques
- Cross-domain knowledge transfer

2. Advanced Ensemble Methods

- 5+ retrieval model combination
- Dynamic weight optimization
- Confidence-based model selection

3. Query Understanding

- Multi-hop reasoning detection
- Complex query decomposition
- Intent-aware retrieval

4. Scalable Architecture

- GPU-optimized processing
 - Distributed retrieval system
 - Real-time performance monitoring
-

EXECUTION CHECKLIST

Daily Tasks

- ☐ Code commits with detailed logs
- ☐ Performance metrics tracking
- ☐ Error analysis and fixes
- ☐ Literature review (30 min/day)

Weekly Reviews

- ☐ Milestone achievement verification
- ☐ Performance comparison with baselines
- ☐ Resource utilization optimization
- ☐ Risk assessment and mitigation

Critical Success Factors

1. **Maximize Recall** - Most important metric (50% weight)

2. **Perfect Ranking** - NDCG optimization crucial
 3. **Robust Evaluation** - Prevent overfitting to validation
 4. **Computational Efficiency** - 2-hour demo constraint
 5. **Reproducibility** - Docker containers and version control
-

IMMEDIATE ACTION ITEMS (Next 7 Days)

1. **Set up development environment** with GPU support
2. **Download all training datasets** and verify integrity
3. **Implement basic evaluation metrics** (Precision@k, Recall@k, NDCG)
4. **Create data preprocessing pipeline**
5. **Build BM25 baseline** for initial benchmarking
6. **Set up experiment tracking** with Weights & Biases
7. **Plan resource requirements** (compute, storage, team)

This roadmap is designed to create a TOP-TIER retrieval system that will dominate the competition. Follow it religiously, and you'll be in the top 3 guaranteed! 🏆